

Compiler broken right now, working on new feature
Here is example TAC

```
// checkPositive function
```

```
FUNC checkPositive:
```

```
    t1 = x > 0
```

```
    IF t1 GOTO L1
```

```
    GOTO L2
```

```
L1: RETURN 1
```

```
L2: RETURN 0
```

```
END_FUNC
```

```
// sumRange function
```

```
FUNC sumRange:
```

```
    sum = 0
```

```
    i = 0
```

```
L3: t2 = i < n
```

```
    IF_FALSE t2 GOTO L4
```

```
    t3 = sum + i
```

```
    sum = t3
```

```
    t4 = i + 1
```

```
    i = t4
```

```
    GOTO L3
```

```
L4: RETURN sum
```

```
END_FUNC
```

```
// main function
```

```
FUNC main:
```

```
    PARAM 5
```

```
    t5 = CALL checkPositive, 1
```

```
    result = t5
```

```
    PARAM 10
```

```
    t6 = CALL sumRange, 1
```

```
    total = t6
```

```
    RETURN 0
```

```
END_FUNC
```

Three key optimizations applied are: (1) Constant Propagation and Folding - replacing PARAM 5 and the call with direct computation since checkPositive(5) always returns 1, and partially evaluating sumRange(10) to compute the result 45 at compile time; (2) Copy Propagation - eliminating temporary variables like t3, t4, t5, t6 by directly using the expressions (e.g., sum = sum + i instead of t3 = sum + i; sum = t3); (3) Dead Code Elimination - removing unused intermediate assignments and unreachable code paths. After optimization, the intermediate code becomes:

FUNC checkPositive:

IF x > 0 GOTO L1

RETURN 0

L1: RETURN 1

END_FUNC

FUNC sumRange:

sum = 0

i = 0

L3: IF i >= n GOTO L4

sum = sum + i

i = i + 1

GOTO L3

L4: RETURN sum

END_FUNC

FUNC main:

result = 1

total = 45

RETURN 0

END_FUNC

The optimized version demonstrates significant improvements: the main function now directly assigns constant values (result=1, total=45) rather than making function calls, reducing execution time from multiple function invocations to simple immediate value assignments.